

Egison Core

Syntax and Typing Rules

本仕様の機械化の最上位目標は、Egison core の注釈不要性 (annotation-freeness), すなわち閉じたプログラムに対する受理完全性 $\forall e \tau, \emptyset \vdash e : \tau \Rightarrow \text{infer}(e) \neq \text{none}$ である。core の構文には型注釈の形が存在しないため、これが「正しいプログラムに注釈は不要」の正確な形である。この命題は完成後の推論器に対する到達目標である。最初の実例だった or pattern の分岐別 freshening は、binder 名の照合と束縛型の単一化による整合へ改めて解消し、受理側の regression として固定した。残る既知の受理ギャップのうち、入れ子 matcher capability の rigid 比較は機械化済みの反例で固定した：wildcard slot 域の単相 consumer に bare ‘something’ と product lift を順に渡すプログラムは宣言的に型付くが、推論器は第一用法で域を raw な matcher 型に固定し、第二用法の lift された capability を注釈として等値比較して拒否する（逆順も raw-head 死角で拒否）。instance capability の producer guard による固定も機械化済みで固定した：Pack : $\forall \kappa \alpha. \text{Matcher } \kappa \alpha \rightarrow \text{Packed}$ に対する Pack something は宣言的には $\kappa := \text{Any}$ の instance で型付くが、推論器は fresh instance capability を protected producer として記録し、guard がその束縛を拒否する（capability を scheme 側で Any に固定した control は受理される）。両反例は現行推論器に対する注釈不要性を反証し、origin-aware な paired unifier で解消させる。達成済みの支柱は次である。

The soundness of executable type inference for the Egison core is mechanized in Lean 4.

ここで soundness は、公開 executable inference が成功すれば、返した型に対する declarative expression typing が必ず構成できることを意味する。機械化した定理は後述する静的入力整形式性を追加前提にしない。completeness は上記の open な目標であり、full principality は機械化済み反例により偽である。

1 構文

1.1 Capability, target type, scheme

$\kappa ::= \text{Any} \mid \chi \mid \hat{s} \mid K \bar{\kappa} \mid (\kappa_1, \dots, \kappa_n)$	$\kappa : \text{Cap}$
$\tau ::= \alpha \mid \hat{a} \mid \mathbf{1} \mid \text{Int} \mid \text{Bool} \mid K \bar{\tau} \mid (\tau_1, \dots, \tau_n) \mid \tau_1 \rightarrow \tau_2$ $\mid \text{Matcher } \kappa \tau \mid \text{MatcherSlot } \kappa \tau$	$\tau : \text{Type}$
$\sigma ::= \forall(\bar{\chi} : \text{Cap}). \forall(\bar{\alpha} : \text{Type}). \tau$	expression scheme
$\delta ::= (\kappa_1 \triangleright \tau_1), \dots, (\kappa_n \triangleright \tau_n) \Rightarrow (\kappa \triangleright \tau)$	pattern-function dual
$\varsigma ::= \forall(\bar{\chi} : \text{Cap}). \forall(\bar{\alpha} : \text{Type}). \delta$	dual scheme
$\eta ::= \kappa \triangleright \tau$	hole descriptor.

図1 二 sort の型構文。 χ と α は flexible meta-variable である。 $\hat{s}$ と $\hat{a}$ の rigid skolem は、明示的な全称変数を型検査中だけ新鮮な固定定数として扱ったものであり、unification で別の capability / 型へ置換できない。 $\text{mono } \tau$ は量化変数をもたない単相 scheme を表し、環境から取り出して instantiate しても新しい変数を作らず同じ τ を返す。

$$\begin{aligned}
e &::= x \mid \lambda x. e \mid e_1 e_2 \mid \text{let } x = e_1 \text{ in } e_2 \mid \text{fix } f \ x. e \mid n \\
&\quad \mid (e_1, \dots, e_n) \mid C \bar{e} \mid \text{op}(\bar{e}) \mid \text{something} \\
&\quad \mid \text{matcher } cls \\
&\quad \mid \text{matchAll } e_t \text{ as } e_m \text{ with } p \rightarrow e \\
&\quad \mid \text{match } e_t \text{ as } e_m \text{ with } (p_i \rightarrow e_i)_i \\
p &::= \$x \mid _ \mid \#e \mid \tilde{x} \mid c \bar{p} \mid p_1 \& p_2 \mid p_1 \mid p_2 \mid f \bar{p} \mid (p_1, \dots, p_n) \\
pp &::= \$ \mid _ \mid \# \$x \mid c \bar{pp} \mid (pp_1, \dots, pp_n) \\
dp &::= \$x \mid _ \mid C \bar{dp} \mid (dp_1, \dots, dp_n) \\
cl &::= pp \text{ as } M \text{ with } [dp_j \rightarrow N_j]_{j=1}^r \\
cls &::= [cl_i]_{i=1}^m \\
d &::= \text{def pattern } f(x_1 : \tau_1, \dots, x_n : \tau_n) := p \\
\ell &::= \text{Any} \mid \chi \mid \hat{s} \\
ev &::= \text{unseen} \mid \text{known } \ell \mid K \bar{ev} \mid (ev_1, \dots, ev_n).
\end{aligned}$$

図2 c は pattern constructor, C は data constructor である。match は matchAll と head への surface elaboration であり, core primitive ではない。

1.2 Source forms

1.3 Contexts and judgments

$\hat{\Sigma} = (\hat{\Sigma}_D, \hat{\Sigma}_P, \hat{\Sigma}_F, \hat{\Sigma}_{prim}, Obs_{\hat{\Sigma}})$	freeze 済み canonical signature
$\Gamma ::= \emptyset \mid \Gamma, x : \sigma$	expression scheme context
$\Phi ::= \emptyset \mid \Phi, x : \text{Pattern}(\kappa \triangleright \tau)$	pattern-parameter dual context
$\Delta ::= \emptyset \mid \Delta, x : \tau$	monomorphic pattern-variable context.

$\hat{\Sigma}; \Gamma \vdash e : \tau$	expression typing
$\hat{\Sigma}; \Gamma; \Phi; \Delta \vdash p : \text{Pattern}(\kappa \triangleright \tau); \Delta'$	pattern dual typing
$\hat{\Sigma} \vdash pp : \text{PPPattern } \tau \rightsquigarrow \bar{\eta}; \Delta_{pp}$	primitive-pattern hole/capture extraction
$\hat{\Sigma}_D \vdash dp : \text{PDPattern } \tau \rightsquigarrow \Gamma_{dp}$	data-pattern binding
$\hat{\Sigma}; \Gamma \vdash cl : \text{Clause } \kappa \tau \rightsquigarrow ev$	matcher-clause typing and evidence
$\hat{\Sigma}; \Gamma \vdash d \text{ ok} \Rightarrow (f : \varsigma)$	pattern-function definition

図3 Primary judgments. Δ と Γ_{dp} を Γ に入れるときは各 binding を $\text{mono } \tau$ として扱う。 $\uplus$ は domain-disjoint union, pattern の Δ は左から右へ thread する。

$\Gamma(x)$, $\Phi(x)$, $\hat{\Sigma}_D(C)$ などは key が重複しない有限 map の lookup であり, key が存在しなければ失敗する。特に $\hat{\Sigma}_F$ の pattern-function 名は相異なる。“fresh” は現在の signature, context, 型, capability に自由出現しない同じ sort の変数を選ぶことを表す。scheme 内の量化 binder は局所的であり, 無関係な fresh 選択の ambient scope には含まない。

2 補助定義

この節の「部分関数」は成功値または失敗を返す。 $f(x) = \text{some } y$ と $f(x) \Downarrow y$ はともに成功値が y であることを表し, $f(x) = \text{none}$ は失敗を表す。判断規則の premise に現れる部分関数が失敗した場合, その規則は適用できない。

2.1 Instantiation, generalization, paired substitution

Capability substitution C と target substitution T は別 sort であり, $S = (C, T)$ の作用を次で定める。

$$S\kappa = C\kappa, \quad S\tau = T(C\tau), \quad S\Gamma = \{x : S\sigma \mid x : \sigma \in \Gamma\}.$$

Paired substitution $S = (C, T)$ は、すべての $\alpha \in \text{dom}(T)$ について $C(T\alpha) = T\alpha$ を満たすとき range-fixed である。この条件は raw な `matchCap` / `mguTy` certificate と、Algorithm W が保持する最初の fresh allocation に対して用いる。一方、scheme binder の局所 mapping と、その結果に対する target specialization は、名前の数値が偶然一致しても局所 binder が後段の型に入り込んだ capability を捕捉しないよう、一つの global pair に潰さず順に適用する。

$$\begin{aligned}
\text{ValueFlowInst}(\forall \bar{\chi}. \forall \bar{\alpha}. \tau) &= T_s(C_v \tau), \\
&\quad \text{dom}(C_v) \subseteq \bar{\chi}, \quad C_v \bar{\chi} \text{ capability variables,} \\
&\quad \text{dom}(T_s) \subseteq \bar{\alpha}, \\
\text{ValueFlowInst}(\text{mono } \tau) &= \tau, \\
\text{Gen}(\hat{\Sigma}, \Gamma, \tau) &= \forall (\text{fcv}(\tau) \setminus (\text{fcv}(\hat{\Sigma}) \cup \text{fcv}(\Gamma)) : \text{Cap}). \\
&\quad \forall (\text{ftv}(\tau) \setminus (\text{ftv}(\hat{\Sigma}) \cup \text{ftv}(\Gamma)) : \text{Type}). \tau, \\
\text{GenDual}(\hat{\Sigma}, \Gamma, \delta) &= \forall \text{shared}(\delta; A) : \text{Cap}. \\
&\quad \forall (\text{ftv}(\delta) \setminus (\text{ftv}(\hat{\Sigma}) \cup \text{ftv}(\Gamma)) : \text{Type}). D_{\text{single}(\delta; A)} \delta,
\end{aligned}$$

where

$$A = \text{fcv}(\hat{\Sigma}) \cup \text{fcv}(\Gamma), \quad \text{single}(\delta; A) = \{\chi \notin A \mid \text{occ}_\delta(\chi) = 1\}, \quad \text{shared}(\delta; A) = \{\chi \notin A \mid \text{occ}_\delta(\chi) \geq 2\},$$

and $D_X \chi = \text{Any}$ for $\chi \in X$, while D_X is the identity on every other capability variable. The occurrence count is taken over all argument and result capabilities, including capability occurrences nested in their target types. Thus an ambient variable is never defaulted or quantified, an independent local leaf has the canonical demand Any, and two or more occurrences keep one quantified variable and hence their sharing. `ValueFlowInst` と `DualValueFlowInst` の宣言的 relation は、capability binder を flexible capability variable へ置換し、その結果に target binder だけを support とする T_s を適用する。 $C_v \chi$ は常に capability variable であり、 $K \bar{\kappa}$ や capability product へ置換してはならない。 T_s は capability variable を変更しない。この制限により、value producer の capability は利用側の consumer demand によって強化されない。 T_s は capability 置換の後にだけ作用する。従って、 T_s が挿入した型の中に binder と同じ番号の自由 capability variable があっても、それを C_v で再び置換しない。この ordered binder-local semantics により、量化 capability binder の variable-only instantiation と利用側から来る自由 capability を区別する。宣言的 relation では binder の像の相異性や ambient freshness を要求しない。Algorithm W はより強い allocation witness として、全 binder へ互いに異なる fresh variable を割り当て、solver が後続の target specialization を記録する。その allocation 証拠を忘却して target substitution を合成すれば、上の宣言的 instance が得られる。 `DualValueFlowInst` は dual scheme の両 binder 列を対象にする。 `Gen` と `GenDual` は signature と context の自由変数を量化しない。 `GenDual` はさらに、ambient でない capability variable を dual payload 全体の出現回数で分類し、一回だけのものを Any に default して、二回以上のものだけを量化する。その型引数には prevailing paired substitution を適用し、未解決の inference placeholder を含まない finalized type/capability を用いる。prevailing paired substitution とは、同じ導出でそれ以前に得た全 constraint を解く累積的な一つの (C, T) である。Skolem は generalize しない。 $\text{fcv}(X)$ と $\text{ftv}(X)$ はそれぞれ X に自由出現する flexible capability meta-variable と flexible target meta-variable の集合であり、scheme の binder に束縛された変数を含まない。 $\text{dom}(X)$ は context または substitution X の定義域である。

上の variable-only ordered instance を使うのは expression context lookup と pattern-function lookup である。Algorithm W はそれを実装するとき fresh allocation witness を記録する。一方、data constructor, pattern constructor, primitive operation の `Inst` は binder-supported な二 sort substitution による通常の structural instance であり、capability image を variable に限定しない。これらは value producer の context lookup ではないため、 `ValueFlowInst` の producer-strengthening 制限とは別である。

$$\begin{aligned}
\text{Inst}(\hat{\Sigma}_D(C)) &= \tau_1 \times \cdots \times \tau_n \rightarrow \tau, \\
\text{Inst}(\hat{\Sigma}_P(c)) &= (\tau_1, \dots, \tau_n) \Rightarrow_P \tau, \\
\text{DualValueFlowInst}(\hat{\Sigma}_F(f)) &= (\eta_1, \dots, \eta_n) \Rightarrow \eta, \\
\text{Inst}(\hat{\Sigma}_{\text{prim}}(op)) &= \tau_1 \times \cdots \times \tau_n \rightarrow \tau.
\end{aligned}$$

2.2 Equality, capability matching, signature operations

$$\begin{aligned} \text{mguTy}(\tau, \tau') &= T \quad \text{target-sort deterministic unifier,} \\ \text{mguCap}(\kappa, \kappa') &= C \quad \text{capability-sort deterministic unifier,} \\ \text{matchCap}(\kappa_m, \kappa_p) &= C \quad \text{dom}(C) \subseteq \text{fcv}(\kappa_p), \quad C\kappa_m = \kappa_m, \quad \text{DemandMatches}_C(\kappa_m, \kappa_p). \end{aligned}$$

DemandMatches_C は元の consumer 構文に対して再帰的に定義する。明示的な $\text{DemandMatches}_C(\kappa_m, \text{Any})$ は任意の producer について成立し、 $\text{DemandMatches}_C(\kappa_m, \chi)$ は $C\chi = \kappa_m$ を要求する。同名 constructor, product, skolem は同じ head / arity で子を pointwise に照合する。特に consumer variable が一度 Any に束縛されても、その後の同じ variable は stored value Any との strict equality を要求し、wildcard には変わらない。matchCap は producer κ_m を rigid に読み、consumer demand κ_p の変数だけを解く sound で決定的な one-way matcher であり、条件を満たす substitution がなければ失敗する。mguTy と mguCap はそれぞれの sort の決定的な部分関数であり、成功時は両端を等しくする sound な unifier を返す。constructor / arity の不一致, occurs check の失敗, または rigid skolem を解く必要がある場合は失敗する。ここで主張するのは成功結果の soundness, 関数としての決定性, 置換普遍性 (任意の別 unifier が成功結果を経由して factor すること), fuel 単調性 (成功はより大きい任意の fuel で同じ substitution のまま保存されること), および可解な入力に対する $\exists \text{fuel}$ 完全性 (unifier を持つ制約はある fuel で成功し, 可解性と $\exists \text{fuel}$ 成功が同値になること) である。固定の構造 fuel bound を使う wrapper の十分性は含まない。Any はこの一方向 consumer relation でだけ wildcard であり, 対称な mguCap では通常の rigid ground constructor として扱う。従って $\text{mguCap}(\text{Any}, \text{Any})$ は成功するが, $\text{mguCap}(\text{Any}, K\bar{\kappa})$ は失敗する (flexible variable を Any へ束縛することはできる)。Frozen pattern signature が提供する target instance と, capability projection の整合判断を次で表す。

$$\pi_c = \text{Inst}(\widehat{\Sigma}_P(c)), \quad \text{CapCompatible}_{\widehat{\Sigma}}(\pi_c; \bar{\kappa}, \kappa), \quad \text{ty}_{\widehat{\Sigma}}(\pi_c; \bar{\tau}) \Downarrow \tau'.$$

π_c はこの二つの判断へ明示的に渡す同一の fresh pattern-constructor signature instance である。CapCompatible はその field に $\bar{\kappa}$ の canonical evidence を対応付け, result argument slot へ到達する evidence だけを部分 evidence ev_p として投影する (result-variable evidence projection)。その上で,

$$\text{merge}(ev_p, [\kappa]_{ev}) = \text{some } [\kappa]_{ev}$$

となるときに限り成立する。projection 自体が完全 capability を返すとは限らない。たとえば nullary nil は observable な list element parameter を unseen のまま残し, 周囲の最終 matcher capability がその位置を埋める。ty は $\bar{\tau}$ を field target と照合し, 同じ signature instance の result target を返す。いずれも arity, canonical former, または rigid constraint が一致しなければ失敗する。CapCompatible の projection は $\text{Obs}_{\widehat{\Sigma}}$ が公開する signature path だけをたどり, target substitution や expected target から capability を生成しない。

2.3 Matcher-clause metafunctions

$$\begin{aligned} \text{prod}_0() &= (), \\ \text{prod}_1(\tau_1) &= \tau_1, \\ \text{prod}_k(\tau_1, \dots, \tau_k) &= (\tau_1, \dots, \tau_k) \quad (k \geq 2), \\ \text{decomposeME}(), 0 &= [], \\ \text{decomposeME}(M, 1) &= [M], \\ \text{decomposeME}((M_1, \dots, M_k), k) &= [M_1, \dots, M_k] \quad (k \geq 2). \end{aligned}$$

上記以外の decomposeME は失敗する。() は nullary tuple value であり, その型も nullary product () である。1 とは同一視しない (この core では 1 の runtime constructor を持たない)。したがって $k = 0$ の matcher expression は () だけである。

Complete capability を partial evidence へ埋め込む canonical map は次である.

$$\begin{aligned} [\text{Any}]_{ev} &= \text{known Any}, & [\chi]_{ev} &= \text{known } \chi, \\ [\hat{s}]_{ev} &= \text{known } \hat{s}, & [K\bar{\kappa}]_{ev} &= K [\bar{\kappa}]_{ev}, \\ [(\kappa_1, \dots, \kappa_n)]_{ev} &= ([\kappa_1]_{ev}, \dots, [\kappa_n]_{ev}). \end{aligned}$$

$\text{clauseEvidence}_{\hat{\Sigma}}(pp; [\kappa_1, \dots, \kappa_k])$ は, pp の hole を左から右へ並べた列と, それぞれの hole に実際に渡される next matcher の slot capability 列から, 一 clause 分の partial evidence を返す決定的な部分関数である. bare-hole catch-all $\$$, wildcard $_$, value pattern $\#\$x$ は root evidence を生成せず **unseen** を返す一方, constructor または tuple の内側にある $\$$ は対応する κ_i の canonical evidence を供給する. Tuple は component evidence をそのまま product node にし, constructor-headed pp は fresh/frozen signature の field type に対して actual hole evidence の observable な structured head / arity を検査してから, result argument slot に到達する部分だけを投影する. このとき surface constructor 名ではなく signature の result former を evidence の root とする. hole 数, arity, canonical former, または同じ result slot へ届く evidence が exact に一致しなければ **none** を返す. target 型, expected 型, annotation は evidence の seed にしない. **unseen** は field-head obligation も assignment も作らず, opaque / function / unobservable path は barrier として内部を観測しない. non-unseen evidence の result variable へ到達しない closed path も assignment は作らないが, その observable な constructor / product head の検査は省略しない. 必要な observable path に既知の異なる head が現れた場合は, **unseen** へ弱めず失敗する. さらに $\text{PPatCoreOrder}(pp)$ は pp を depth-first ・ 左から右に走査し, hole $\$$ を一度訪れた後には value-pattern-pattern $\#\$x$ が現れないことを表す. clauseEvidence はこの条件を最初に検査し, 満たさなければ **none** を返す. Algorithm W は matcher literal の raw finalization でこの evidence 計算を行うため, 逆順の clause は inferRaw の時点で棄却される. さらに terminal validator は最終 substitution の下で同じ clause evidence を再計算し, public cut に第二の fail-closed な検査を置く. 従って $\text{cons } \$ \#\x や $(\$, \#\$x)$ は formal core では不受理だが, $\text{cons } \#\$x \$$ は受理される. 順序を逆にした pattern constructor を用意すれば同じ matching を表現できるため, これは表現力ではなく core の安全な正規形に関する制限である. Egison 本体では利便性のため, この逸脱を `--pattern-hole-before-primitive-value-pattern-warnings` の警告として扱ってよい. 従って $\text{box} : (\text{List Int}) \Rightarrow_P \text{Box}$ の closed field は result capability へ evidence を投影しないが, $\text{box } \$$ の actual next matcher capability は list head を持たなければならない. 一方, generic な CapCompatible / projection 自体にはこの actual-hole 検査を加えないので, $\text{box } _$ や $\text{box } \#\$x$ は不要な hole obligation を受けない. $\text{Matcher Any List Int}$ という型それ自体は有効であり, 却下されるのはその matcher をこの closed list hole の actual producer として使う場合だけである.

$\text{PPatCapsAt}_{\hat{\Sigma}}(b, pp, \bar{\kappa}, \kappa)$ は, pp の hole 列を $\bar{\kappa}$ と左から右へ対応させ, その構造が最終 matcher capability κ と整合することを表す. nested hole はその位置の capability と同じでなければならない. constructor node は CapCompatible , tuple node は capability product を使う. root hole だけは catch-all として任意の一 hole capability を消費でき, wildcard と value pattern は hole を消費しない.

ClausesTy は actual clause list を順に検査し, 一 clause につき一つの evidence を同じ順序で返す output-indexed judgment である. 全 clause は同じ最終 capability と target を共有する.

$$\text{ClausesTy}(\hat{\Sigma}; \Gamma, [cl_i]_{i=1}^m, \kappa, \tau) \Downarrow [ev_i]_{i=1}^m \iff \forall i. \hat{\Sigma}; \Gamma \vdash cl_i : \text{Clause } \kappa \tau \rightsquigarrow ev_i.$$

いずれかの clause の PP extraction, next-matcher decomposition / slot typing, または一 clause 分の evidence projection が失敗すれば, この判断は成立しない.

2.4 Shape and coverage

Evidence の exact merge と shape inference は次で定める.

$$\begin{aligned} \text{merge}(\text{unseen}, ev) &= ev & \text{merge}(ev, \text{unseen}) &= ev, \\ \text{merge}(\text{known } \ell, \text{known } \ell) &= \text{known } \ell, & \text{merge}(K\bar{e}, K\bar{f}) &= K \text{merge}(\bar{e}, \bar{f}), \\ \text{merge}((\bar{e}), (\bar{f})) &= (\text{merge}(\bar{e}, \bar{f})). \end{aligned}$$

Constructor name, node kind, arity, leaf identity が一致しない merge は失敗する. 上の式では成功時の `some` を省略している. 列上の merge は同じ長さの列を pointwise に merge し, 長さが異なるか一成分でも失敗すれば失敗する. `mergeAll` は `unseen` を初期値として evidence list を merge で畳み込む決定的な部分関数であり, 途中の merge が一度でも失敗すれば `none` を返す.

$$\text{inferShape}(Obs, \overline{ev}) = \begin{cases} \text{some Any} & \text{mergeAll}(\overline{ev}) = \text{some unseen}, \\ \text{finalize}_{Obs}(ev) & \text{mergeAll}(\overline{ev}) = \text{some } ev, \text{ } ev \neq \text{unseen}, \\ \text{none} & \text{mergeAll}(\overline{ev}) = \text{none}. \end{cases}$$

`finalizeObs` は partial evidence を complete capability へ変換する決定的な部分関数である. observable な `unseen` が残れば `none` を返し, unobservable な constructor parameter は Any に canonicalize する. known Any, known χ , known $\hat{s}$ はそれぞれ Any, χ , $\hat{s}$ に戻す. Constructor node は root former と同じ長さの frozen observability mask を用い, product component はすべて observable として再帰的に finalize する. 未知の mask や arity 不一致は失敗する.

$$\text{ShapeCap}_{\hat{\Sigma}}(\overline{ev}, \kappa) \iff \text{inferShape}(Obs_{\hat{\Sigma}}, \overline{ev}) = \text{some } \kappa.$$

T-MATCHER は `ClausesTy` が actual clause list から返した同じ $\overline{ev}$ を `ShapeCap` へ渡すため, 別の evidence list を選び直すことはできない. $Obs_{\hat{\Sigma}}(K)$ は canonical former K の parameter 数と同じ長さの Boolean mask を返し, 各 parameter が pattern-constructor signature から capability として観測可能かを示す. この mask は freeze 前に全 signature から計算した least fixpoint であり, 個々の matcher clause や target annotation には依存しない.

$$\begin{aligned} \text{CoverageOK}_{\hat{\Sigma}}(cls, \text{Any}) &\iff \text{true}, \\ \text{CoverageOK}_{\hat{\Sigma}}(cls, K\bar{\kappa}) &\iff \exists \bar{c}. \text{constructorsFor?}_{\hat{\Sigma}_P}(K) = \text{some } \bar{c} \wedge \forall c \in \bar{c}. \exists cl \in cls. cl.pp = c \$ \dots \$, \\ \text{CoverageOK}_{\hat{\Sigma}}(cls, (\kappa_1, \dots, \kappa_n)) &\iff \exists cl \in cls. cl.pp = (\$, \dots, \$). \end{aligned}$$

`CoverageOK` のその他の root form は成立しない. Refinement は general clause に数えず, coverage は child capability へ再帰しない. `constructorsFor?_{\hat{\Sigma}_P}(K)` は独立に freeze した保守的 coverage-index row の lookup であり, $cl.pp$ は clause cl の primitive-pattern-pattern を取り出す selector である. `FrozenSigWF` checker は, 互換な各 pattern-constructor instance が canonical root former の row に正しい arity で含まれ, その row が実在することを保証する. row 内の追加 entry は許されるが, 要求される general clause を増やして coverage を厳しくするだけである. Formal core では `CoverageOK` を matcher literal の必須条件とする. Egison 本体は同じ検査の失敗を warning として扱ってよいが, その診断方針は core の外側に置く.

2.5 Matcher well-formedness predicates

$$\begin{aligned} \text{CatchAllLast}(cls) &\iff \exists \text{prefix}, M, x, N. cls = \text{prefix} ++ [\$ \text{ as } M \text{ with } [\$ x \rightarrow N]] \\ &\quad \wedge \forall cl \in \text{prefix}. cl.pp \neq \$, \\ \text{ArmExhaustive}_{\hat{\Sigma}_D}(cls, \tau) &\iff \forall cl \in cls. \text{Exhaustive}_{\hat{\Sigma}_D}(\text{arms}(cl), \tau), \\ \text{PPBindNodup}(cls) &\iff \forall cl \in cls. \text{NoDup}(\text{bindPP}(cl.pp)), \\ \text{ArmBindNodup}(cls) &\iff \forall cl \in cls. \forall dp \in \text{arms}(cl). (\text{NoDup}(\text{bindDP}(dp)) \\ &\quad \wedge \text{bindDP}(dp) \cap \text{bindPP}(cl.pp) = \emptyset). \end{aligned}$$

$\text{arms}(cl)$ は clause の $[dp_j \rightarrow N_j]_j$ から data pattern 列 $[dp_j]_j$ を順序を保って取り出す. `bindPP(pp)` と `bindDP(dp)` はそれぞれ pp の $\# \$ x$ と dp の $\$ x$ が導入する変数名を左から右へ集め, `NoDup` はその列に重複がないことを表す. $\text{Exhaustive}_{\hat{\Sigma}_D}(\overline{dp}, \tau)$ は保守的で実行可能な checker である. core では $\overline{dp}$ に変数 pattern または wildcard が一つでもあれば成功し, それ以外は失敗する. したがって成功時には任意の runtime value を受理する具体的な arm が存在する. constructor の網羅性を推測する外部 oracle は用いない.

3 Expression typing

$$\begin{array}{c}
\frac{\Gamma(x) = \sigma \quad \text{ValueFlowInst}(\sigma) = \tau}{\widehat{\Sigma}; \Gamma \vdash x : \tau} \quad [\text{T-VAR}] \\
\frac{\widehat{\Sigma}; \Gamma, x : \text{mono } \tau_1 \vdash e : \tau_2}{\widehat{\Sigma}; \Gamma \vdash \lambda x. e : \tau_1 \rightarrow \tau_2} \quad [\text{T-LAM}] \\
\frac{\widehat{\Sigma}; \Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \widehat{\Sigma}; \Gamma \vdash e_2 : \tau_1}{\widehat{\Sigma}; \Gamma \vdash e_1 e_2 : \tau_2} \quad [\text{T-APP}] \\
\frac{}{\widehat{\Sigma}; \Gamma \vdash n : \text{Int}} \quad [\text{T-LIT}] \\
\frac{\widehat{\Sigma}; \Gamma \vdash e_1 : \tau_1 \quad \sigma = \text{Gen}(\widehat{\Sigma}, \Gamma, \tau_1) \quad \widehat{\Sigma}; \Gamma, x : \sigma \vdash e_2 : \tau_2}{\widehat{\Sigma}; \Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2} \quad [\text{T-LET}] \\
\frac{f \neq x \quad \text{DirectSelf}(f, e) \quad \widehat{\Sigma}; \Gamma, f : \text{mono } (\tau_1 \rightarrow \tau_2), x : \text{mono } \tau_1 \vdash e : \tau_2}{\widehat{\Sigma}; \Gamma \vdash \text{fix } f x. e : \tau_1 \rightarrow \tau_2} \quad [\text{T-FIX}] \\
\frac{\widehat{\Sigma}; \Gamma \vdash e_i : \tau_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma \vdash (e_1, \dots, e_n) : (\tau_1, \dots, \tau_n)} \quad [\text{T-TUPLE}]
\end{array}$$

$\text{DirectSelf}(f, e)$ は shadowing-aware な決定可能構文条件である。 f の自由出現は application spine の head に直接現れる場合だけを受理し、bare value, 引数位置, alias 経由, 高階の受渡しを拒否する。従って T-FIX 自身が公開 inference と同じ singleton direct-self 境界を持つ。

$$\begin{array}{c}
\frac{\text{Inst}(\widehat{\Sigma}_D(C)) = \tau_1 \times \dots \times \tau_n \rightarrow \tau \quad \widehat{\Sigma}; \Gamma \vdash e_i : \tau_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma \vdash C e_1 \dots e_n : \tau} \quad [\text{T-CON}] \\
\frac{\text{Inst}(\widehat{\Sigma}_{\text{prim}}(op)) = \tau_1 \times \dots \times \tau_n \rightarrow \tau \quad \widehat{\Sigma}; \Gamma \vdash e_i : \tau_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma \vdash op(e_1, \dots, e_n) : \tau} \quad [\text{T-PRIM}] \\
\frac{}{\widehat{\Sigma}; \Gamma \vdash \text{something} : \text{Matcher Any } \tau} \quad [\text{T-SOME}]
\end{array}$$

T-SOME の τ は任意である。fresh variable を選ぶのは Algorithm W の実装上の一手段だが、宣言的 judgment は substitution に閉じた形を直接採用する。

4 Pattern typing and pattern functions

$$\begin{array}{c}
\frac{x \notin \text{dom}(\Delta) \quad \chi, \alpha \text{ fresh}}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash \$x : \text{Pattern}(\chi \triangleright \alpha); \Delta, x : \alpha} \quad [\text{PAT-VAR}] \\
\frac{\chi, \alpha \text{ fresh}}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash _ : \text{Pattern}(\chi \triangleright \alpha); \Delta} \quad [\text{PAT-WILD}] \\
\frac{\widehat{\Sigma}; \Gamma, \text{mono } \Delta \vdash e : \tau \quad \chi \text{ fresh}}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash \#e : \text{Pattern}(\chi \triangleright \tau); \Delta} \quad [\text{PAT-VALUE}] \\
\frac{\Phi(x) = \text{Pattern}(\kappa \triangleright \tau)}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash \widetilde{x} : \text{Pattern}(\kappa \triangleright \tau); \Delta} \quad [\text{PAT-EMBED}]
\end{array}$$

$$\frac{\widehat{\Sigma}; \Gamma; \Phi; \Delta_{i-1} \vdash p_i : \text{Pattern}(\kappa_i \triangleright \tau_i); \Delta_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma; \Phi; \Delta_0 \vdash (p_1, \dots, p_n) : \text{Pattern}((\kappa_1, \dots, \kappa_n) \triangleright (\tau_1, \dots, \tau_n)); \Delta_n} \quad [\text{PAT-TUPLE}]$$

$$\frac{\pi_c = \text{Inst}(\widehat{\Sigma}_P(c)) = (\rho_1, \dots, \rho_n) \Rightarrow_P \rho \quad \widehat{\Sigma}; \Gamma; \Phi; \Delta_{i-1} \vdash p_i : \text{Pattern}(\kappa_i \triangleright \tau_i); \Delta_i \quad (1 \leq i \leq n) \quad \text{CapCompatible}_{\widehat{\Sigma}}(\pi_c; \bar{\kappa}, \kappa) \quad \text{ty}_{\widehat{\Sigma}}(\pi_c; \bar{\tau}) \Downarrow \tau}{\widehat{\Sigma}; \Gamma; \Phi; \Delta_0 \vdash c p_1 \cdots p_n : \text{Pattern}(\kappa \triangleright \tau); \Delta_n} \quad [\text{PAT-CON}]$$

$$\frac{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_1 : \text{Pattern}(\kappa \triangleright \tau); \Delta_1 \quad \widehat{\Sigma}; \Gamma; \Phi; \Delta_1 \vdash p_2 : \text{Pattern}(\kappa \triangleright \tau); \Delta_2}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_1 \& p_2 : \text{Pattern}(\kappa \triangleright \tau); \Delta_2} \quad [\text{PAT-AND}]$$

$$\frac{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_1 : \text{Pattern}(\kappa \triangleright \tau); \Delta' \quad \widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_2 : \text{Pattern}(\kappa \triangleright \tau); \Delta'}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_1 \mid p_2 : \text{Pattern}(\kappa \triangleright \tau); \Delta'} \quad [\text{PAT-OR}]$$

$$\frac{\text{DualValueFlowInst}(\widehat{\Sigma}_F(f)) = (\kappa_1 \triangleright \tau_1), \dots, (\kappa_n \triangleright \tau_n) \Rightarrow (\kappa \triangleright \tau) \quad \widehat{\Sigma}; \Gamma; \Phi; \Delta_{i-1} \vdash p_i : \text{Pattern}(\kappa_i \triangleright \tau_i); \Delta_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma; \Phi; \Delta_0 \vdash f p_1 \cdots p_n : \text{Pattern}(\kappa \triangleright \tau); \Delta_n} \quad [\text{PAT-APP}]$$

$$\frac{\begin{array}{l} \widehat{\Sigma}_F(f) = \varsigma_f \quad f \notin \text{pcalls}(p_{\text{body}}) \quad \chi_1, \dots, \chi_n \text{ fresh} \quad \delta_{\text{raw}} = (\chi_1 \triangleright \tau_1), \dots, (\chi_n \triangleright \tau_n) \Rightarrow (\kappa \triangleright \tau) \\ \varsigma^\circ = \text{GenDual}(\widehat{\Sigma} \setminus f, \Gamma, \delta_{\text{raw}}) \quad \text{payload}(\varsigma^\circ) = ((\kappa_1^\circ \triangleright \tau_1^\circ), \dots, (\kappa_n^\circ \triangleright \tau_n^\circ) \Rightarrow (\kappa^\circ \triangleright \tau^\circ)) \\ \Phi^\circ = \emptyset, x_1 : \text{Pattern}(\kappa_1^\circ \triangleright \tau_1^\circ), \dots, x_n : \text{Pattern}(\kappa_n^\circ \triangleright \tau_n^\circ) \quad \exists S, \Delta. \quad \widehat{\Sigma}; \Gamma; \Phi^\circ; \emptyset \vdash_S p_{\text{body}} : \text{Pattern}(\kappa^\circ \triangleright \tau^\circ); \Delta \\ \widetilde{x_1}, \dots, \widetilde{x_n} \text{ occur exactly once, in declaration order, outside or-alternatives} \quad \varsigma_f \equiv_{\text{vf}} \varsigma^\circ \end{array}}{\widehat{\Sigma}; \Gamma \vdash \text{def pattern } f(x_1 : \tau_1, \dots, x_n : \tau_n) := p_{\text{body}} \text{ ok} \Rightarrow (f : \varsigma_f)} \quad [\text{PATFUN-DEF}]$$

ここで $\widehat{\Sigma} \setminus f$ は、有限 map $\widehat{\Sigma}_F$ から一意な f の entry だけを除いた signature である。本体は freeze 済みの完全な $\widehat{\Sigma}$ で検査し、自身の scheme だけを generalization の ambient free-variable 集合から除外する。 δ_{raw} は fresh leaf を保持する inference provenance であり、 ς° の payload はそれに GenDual の singleton-default action を一度だけ適用した canonical core index である。 $\vdash_S$ は、本体全体で一つの prevailing paired substitution S を共有する resolved pattern judgment (Lean の ResolvedPatternTy) を表す。本体 derivation はこの正規化済み parameter / result payload に加えて、raw leaf から terminal index への輸送に用いる S を保持する。この存在する S を、payload を定める canonical な singleton-default action そのものとは同一視しない。以後の DualValueFlowInst は共有 capability の variable-only rename と target specialization だけを行い、Any default を再実行しない。 $\varsigma_1 \equiv_{\text{vf}} \varsigma_2$ は、両 dual scheme が同じ DualValueFlowInst の引数／結果 relation を持ち、自由 capability 変数集合と自由 target 変数集合も等しいことを表す。従って context ごとの fresh allocation に伴う alpha-renaming を含む観測不能な binder 表現差を許すが、許される value flow と ambient free-variable 境界は変えない。pcalls は pattern の全子孫を走査し、value pattern 内の式、その matchAll pattern, matcher clause の next と各 arm body も含めて自己 papp を拒否する。従って他の pattern function への参照は許すが、直接自己再帰は許さない。

5 Match and MatcherSlot

$$\frac{\widehat{\Sigma}; \Gamma \vdash e_t : \tau_t \quad \widehat{\Sigma}; \Gamma; \emptyset; \emptyset \vdash p : \text{Pattern}(\kappa_p \triangleright \tau_t); \Delta \quad \widehat{\Sigma}; \Gamma \vdash e_m : \text{MatcherSlot } \kappa_p \tau_t \quad \widehat{\Sigma}; \Gamma, \text{mono } \Delta \vdash e : \tau_r}{\widehat{\Sigma}; \Gamma \vdash \text{matchAll } e_t \text{ as } e_m \text{ with } p \rightarrow e : \text{List } \tau_r} \quad [\text{T-MATCHALL}]$$

$$\frac{\widehat{\Sigma}; \Gamma \vdash e_t : \tau_t \quad \widehat{\Sigma}; \Gamma \vdash e_m : \text{MatcherSlot } \kappa_i \tau_t \quad \widehat{\Sigma}; \Gamma, \text{mono } \Delta_i \vdash e_i : \tau_r \quad (\forall i)}{\widehat{\Sigma}; \Gamma \vdash \text{match } e_t \text{ as } e_m \text{ with } (p_i \rightarrow e_i)_i : \tau_r} \quad [\text{T-MATCH (DERIVED)}]$$

$$\frac{\text{matchCap}(\kappa_m, \kappa_p) = C \quad \widehat{\Sigma}; \Gamma \vdash e : \text{Matcher } \kappa_m \tau_m \quad \text{mguTy}(C\tau_m, C\tau_t) = T \quad S = (C, T) \quad C(T\alpha) = T\alpha \quad (\alpha \in \text{dom}(T))}{\widehat{\Sigma}; S\Gamma \vdash e : \text{MatcherSlot } C\kappa_p S\tau_t} \quad [\text{COERCE-MATCHER-TO-SLOT}]$$

$$\frac{\text{mguCap}(\kappa', \kappa) = C \quad \widehat{\Sigma}; \Gamma \vdash e : \text{MatcherSlot } \kappa' \tau' \quad \text{mguTy}(C\tau', C\tau) = T \quad S = (C, T) \quad C(T\alpha) = T\alpha \quad (\alpha \in \text{dom}(T))}{\widehat{\Sigma}; S\Gamma \vdash e : \text{MatcherSlot } S\kappa S\tau} \quad [\text{CHECK-SLOT-TO-SLOT}]$$

$$\frac{\widehat{\Sigma}; \Gamma \vdash e : (\text{Matcher } \kappa_1 \tau_1, \dots, \text{Matcher } \kappa_n \tau_n)}{\widehat{\Sigma}; \Gamma \vdash e : \text{Matcher } (\kappa_1, \dots, \kappa_n) (\tau_1, \dots, \tau_n)} \quad [\text{COERCE-PRODUCT-MATCHER}]$$

$$\frac{\widehat{\Sigma}; \Gamma \vdash e : (\text{MatcherSlot } \kappa_1 \tau_1, \dots, \text{MatcherSlot } \kappa_n \tau_n)}{\widehat{\Sigma}; \Gamma \vdash e : \text{MatcherSlot } (\kappa_1, \dots, \kappa_n) (\tau_1, \dots, \tau_n)} \quad [\text{COERCE-SLOT-TUPLE}]$$

6 Matcher literals

6.1 Primitive pattern patterns

$$\frac{\chi \text{ fresh}}{\widehat{\Sigma} \vdash \$: \text{PPPattern } \tau \rightsquigarrow [\chi \triangleright \tau]; \emptyset} \quad [\text{PP-HOLE}] \quad \frac{}{\widehat{\Sigma} \vdash _ : \text{PPPattern } \tau \rightsquigarrow []; \emptyset} \quad [\text{PP-WILD}] \quad \frac{}{\widehat{\Sigma} \vdash \# \$ x : \text{PPPattern } \tau \rightsquigarrow []; x : \tau} \quad [\text{PP-VAL}]$$

$$\frac{\text{Inst}(\widehat{\Sigma}_P(c)) = (\tau_1, \dots, \tau_n) \Rightarrow_P \tau \quad \widehat{\Sigma} \vdash pp_i : \text{PPPattern } \tau_i \rightsquigarrow \bar{\eta}_i; \Delta_i \quad (1 \leq i \leq n) \quad \bar{\eta} = \bar{\eta}_1 ++ \dots ++ \bar{\eta}_n}{\widehat{\Sigma} \vdash c pp_1 \dots pp_n : \text{PPPattern } \tau \rightsquigarrow \bar{\eta}; \Delta_1 \uplus \dots \uplus \Delta_n} \quad [\text{PP-CON}]$$

$$\frac{\widehat{\Sigma} \vdash pp_i : \text{PPPattern } \tau_i \rightsquigarrow \bar{\eta}_i; \Delta_i \quad (1 \leq i \leq n)}{\widehat{\Sigma} \vdash (pp_1, \dots, pp_n) : \text{PPPattern } (\tau_1, \dots, \tau_n) \rightsquigarrow \bar{\eta}_1 ++ \dots ++ \bar{\eta}_n; \Delta_1 \uplus \dots \uplus \Delta_n} \quad [\text{PP-TUPLE}]$$

6.2 Primitive data patterns

$$\begin{array}{c}
\frac{}{\widehat{\Sigma}_D \vdash \$x : \text{PDPattern } \tau \rightsquigarrow x : \tau} \quad [\text{PD-VAR}] \qquad \frac{}{\widehat{\Sigma}_D \vdash _ : \text{PDPattern } \tau \rightsquigarrow \emptyset} \quad [\text{PD-WILD}] \\
\\
\frac{\text{Inst}(\widehat{\Sigma}_D(C)) = \tau_1 \times \cdots \times \tau_n \rightarrow \tau \quad \widehat{\Sigma}_D \vdash dp_i : \text{PDPattern } \tau_i \rightsquigarrow \Gamma_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}_D \vdash C dp_1 \cdots dp_n : \text{PDPattern } \tau \rightsquigarrow \Gamma_1 \uplus \cdots \uplus \Gamma_n} \quad [\text{PD-CON}] \\
\\
\frac{\widehat{\Sigma}_D \vdash dp_i : \text{PDPattern } \tau_i \rightsquigarrow \Gamma_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}_D \vdash (dp_1, \dots, dp_n) : \text{PDPattern } (\tau_1, \dots, \tau_n) \rightsquigarrow \Gamma_1 \uplus \cdots \uplus \Gamma_n} \quad [\text{PD-TUPLE}]
\end{array}$$

6.3 Clauses and matcher construction

$$\begin{array}{c}
\frac{\text{PPatCoreOrder}(pp) \quad \widehat{\Sigma} \vdash pp : \text{PPPattern } \tau \rightsquigarrow [(\kappa_1 \triangleright \tau_1), \dots, (\kappa_k \triangleright \tau_k)]; \Delta_{pp} \quad \text{PPatCapsAt}_{\widehat{\Sigma}}(\text{root}, pp, [\kappa_1, \dots, \kappa_k], \kappa) \quad \text{decomposeME}(M, k) = [M_1, \dots, M_k] \\
\widehat{\Sigma}; \Gamma \vdash M_l : \text{MatcherSlot } \kappa_l \tau_l \quad (1 \leq l \leq k) \quad \widehat{\Sigma}_D \vdash dp_j : \text{PDPattern } \tau \rightsquigarrow \Gamma_j \quad (1 \leq j \leq r) \\
\widehat{\Sigma}; \Gamma, \text{mono } \Delta_{pp}, \text{mono } \Gamma_j \vdash N_j : \text{List prod}_k(\tau_1, \dots, \tau_k) \quad (1 \leq j \leq r) \quad \text{clauseEvidence}_{\widehat{\Sigma}}(pp; [\kappa_1, \dots, \kappa_k]) = \text{some } ev}{\widehat{\Sigma}; \Gamma \vdash (pp \text{ as } M \text{ with } [dp_j \rightarrow N_j]_{j=1}^r) : \text{Clause } \kappa \tau \rightsquigarrow ev} \quad [\text{CLAUSE-TY}] \\
\\
\frac{\text{ClausesTy}(\widehat{\Sigma}; \Gamma, cls, \kappa, \tau) \Downarrow \bar{e}v \quad \text{ShapeCap}_{\widehat{\Sigma}}(\bar{e}v, \kappa) \quad \text{CatchAllLast}(cls) \quad \text{ArmExhaustive}_{\widehat{\Sigma}_D}(cls, \tau) \quad \text{PPBindNodup}(cls) \quad \text{ArmBindNodup}(cls) \quad \text{CoverageOK}_{\widehat{\Sigma}}(cls, \kappa)}{\widehat{\Sigma}; \Gamma \vdash \text{matcher } cls : \text{Matcher } \kappa \tau} \quad [\text{T-MATCHER}]
\end{array}$$

7 機械化する定理の境界

7.1 安全な instance と後続輸送

Inst が使う宣言的 instance と, Algorithm W の allocation witness を区別する. 宣言的な C_v と T_s はそれぞれの scheme binder 上だけに support を持ち, C_v は各 binder を capability variable へ写す. この層では像の相異性, ambient freshness, range-fixedness を使わない. これに対し W の (C_f, T_f) は, 像が互いに異なり ambient scope に対して fresh であることと range-fixedness を記録する. ここで ambient scope は signature と context の自由変数からなり, 別の scheme が局所的に束縛する同名の数値 identifier は含めない. W の witness からこれらの allocation 証拠を忘却すると宣言的 instance が得られる.

Target specialization T_s は capability instance に対して後から作用し, その像に C_v を再適用する global 作用にはまどめない. 証明中で shape evidence を輸送する宣言的な後続 post も capability variable を capability variable にだけ写す. その層では単射性, 像の相異性, ambient freshness を要求しない. 有限 support と fresh allocation を記録する強い restricted chain から忘却することもできるが, 公開再構成が使う target-only suffix の capability 成分は恒等写像なので, 終端 validator は宣言的な variable post を直接構成する. 各 post は現在の導出へ順に作用するため, 前の局所 binder と後から挿入された自由変数を混同しない.

これに対して、raw な `matchCap` / `mguTy` certificate は生成時の正規化済み index と局所 cut とともに保持する。成功 run の終端までに得た solver delta は、後段の capability action を前段の target range 内部にも適用する cross-sort-aware な逐次作用で replay する。終端 validator は、記録した局所 cut と唯一の終端 cut の間で必要な alignment と capability-variable post を検査する。従って誤った componentwise 可換性を仮定せず、solver を別の入力で再実行する必要もない。中間の各 stop を追加の公開前提として量化しない。したがって次が成り立つ。

整型な value producer を scheme から取り出した後、consumer demand, expected target, annotation, または再帰 self-reference を根拠として、その intrinsic capability を構造的に強化することはできない。

primitive-pattern の hole capability は value producer ではなく clause evidence の consumer placeholder である。これらは matcher literal 全体で一つの prevailing substitution により解決し、actual clause evidence と CoverageOK がその構造を根拠づける。両者を同じ instantiation 経路で扱わない。

Let specialization では、bound expression の検査直後に得た context Γ_0 , 型 τ_0 , scheme $\sigma_0 = \text{Gen}(\hat{\Sigma}, \Gamma_0, \tau_0)$ を trace に記録する。body の constraint をすべて反映した終端 substitution S_r に対して validator が検査するのは、次の終端可換式である。

$$S_r \sigma_0 = \text{Gen}(\hat{\Sigma}, S_r \Gamma_{\text{raw}}, S_r \tau_{\text{raw}}).$$

従って、body の検査後も Γ_0 , τ_0 , σ_0 が構文的に不変であるとは仮定しない。外側の instance が導入した自由変数と内側の量化 binder の数値 identifier が一致する場合には、内側の binder を target ambient の外へ局所的に freshen してから輸送する。この freshening は source-level の導出輸送補題である。公開 inference の T-LET bridge は上の終端可換式を validator で直接検査して再構成し、型付け導出そのものを premise とする oracle は置かない。

7.2 検証付き Algorithm W と再構成

raw traversal `inferRaw` は expression, pattern, primitive pattern, data pattern, clause, slot check を有限な fuel 再帰で実行し、成功時には全 solver step と source event を trace に保持する。fresh value instance の capability variable は protected producer として記録し、全 solver delta がそれらを固定することを実行時にも監査する。matcher literal の成功には CoverageOK, catch-all, arm exhaustiveness, binder 線形性の各 checker の成功を要求する。

Pattern constructor の capability inference は、まず zonk 済み child capability に対する exact projection を試みる。子 pattern が互いに独立な fresh consumer capability を持つために result type variable の共有構造がまだ見えず projection が失敗した場合だけ、各 observable result variable に一つの共有 fresh capability を割り当て、constructor の field type へ展開した demand を作る。例えば generic list の `cons` では、二つの field に κ と `List  $\kappa$` を要求する。result variable へ到達しない field は constraint を生成せず、到達する field だけについて child consumer capability と demand の capability equality を protected-producer guard の下で解く。その後に child を再び zonk し、exact projection を再実行する。これは pattern consumer 間の制約解決であり、expected target や value producer capability を seed として構造化する経路ではない。共有 result variable に到達する field の内部では、到達しない observable subposition を Any に canonicalize して field 全体を整合させる保守的な fallback とする。その位置だけを無視する partial/path-wise alignment と W の completeness は主張しない。

executable boundary regression は、独立な child consumer $[?0, ?1]$ に対して旧 exact projection が失敗することから始め、fallback 後には一つの fresh κ を共有して $[\kappa, \text{List } \kappa]$ へ zonk され、result が `List  $\kappa$` となり、最終 CapCompatible が成功することを直接検査する。同時に unrelated protected-producer ledger が不変であり、全 solver delta がその producer を固定することも検査する。さらに target は整合するが capability の rigid head が衝突する pattern constructor を raw/public inference がともに拒否し、capability だけを整合させた control twin が公開 inference と soundness elimination を通ることを対で検査する。量化 pattern-function instance の protected capability variable を実際に `cons` の tail child に置く負例では、raw unifier が structural delta を作れる一方、producer guard がその delta, 直接 fallback, raw inference, public inference をすべて fail closed にすることを固定する。これらは solver / 公開 acceptance 境界の回帰であり、W の completeness 主張ではない。

各 clause が保持する raw hole dual は, literal 全体の prevailing substitution が確定した後に一度だけ zonk する. その時点で全 clause evidence を再計算し, PPatCapsAt の executable checker も最終 capability に対して実行する. 途中の clause で計算した古い evidence を finalization に再利用しない. この再計算では, actual hole evidence に対する closed field-head validation を先に行い, 成功した field のうち result variable へ到達するものだけから result-variable evidence projection により assignment を集める. 従って closed structured field は result evidence を生成しない一方, その next matcher の observable path 上の capability head / arity が field target と一致しなければ inference は失敗する. user pattern constructor についても, 終端 validator は最終 substitution で zonk した child / result capability に対して CapCompatible を再検査する. これらは target や expected type を evidence の seed にしない.

各 nested pattern はそれ自身の fresh raw index を持ち, solver は親が要求する index と終端 substitution 適用後に一致させる. 従って再構成では, 生成時 certificate を残す raw provenance と, 実際の終端 index で構造を追う terminal derivation を分ける. 子の raw index と親の raw field index が構文的に同一であるという, Algorithm W が保証しない条件は要求しない. source typing と runtime proof は terminal derivation の射影だけを利用する.

公開する infer は raw traversal の成功をそのまま公開せず, 有限の終端代数 validator wBridgeCheck を通した結果だけを返す. 概念的な定義は次である.

$$\begin{aligned} \text{infer}(\widehat{\Sigma}, \Gamma, e) = & \text{bind}(\text{inferRaw}(\widehat{\Sigma}, \Gamma, e), \lambda r. \\ & \text{if } \text{wBridgeCheck}(\widehat{\Sigma}, r) \text{ then some } r \text{ else none}). \end{aligned}$$

wBridgeCheck は signature と有限な result / trace だけを走査し, 記録した supply に基づく binder-local instance, slot / type / dual alignment, lookup 時の fresh instance と終端 instance の対応, matcher finalization, 終端 let-generalization, および concrete coverage/exhaustiveness check を検査する. source typing judgment やその導出を検査対象・保存データとして持たない. validator の成功から WBridgeWF を Lean 内で構成し, raw traversal に対する再構成補題へ渡す. solver replay 自体は cross-sort-aware な逐次合成から無条件に導かれる. 元の context / expression と result / trace の対応は raw traversal の成功等式が与える. また公開 soundness の経路では, 生成時の FreshInstAt を後続 solver cut ごとに輸送せず, 終端の binder-local ValueFlowInst / Inst を有限 checker で直接構成する. 従って WBridgeWF は実装内部の中間証明であり, 公開 soundness theorem の caller が与える premise ではない. raw traversal が成功したが終端 validator が失敗した場合, 公開 infer は失敗を返す. この filter が全 typable term または全 raw 成功を受理するという completeness は主張しない.

意図する frozen Egison core 入力 of 静的境界 InferencelInputWF($\widehat{\Sigma}, \Gamma$) は, $\widehat{\Sigma}$ の arm-exhaustiveness 関数が concrete basicArmExhaustive checker であること, および signature と context の lookup から選択可能な各 scheme の capability binder 列と target binder 列がそれぞれ重複を持たないことを要求する. これは runtime safety 用の FrozenSigWF とは別の静的境界である. ただし fail-closed な validator が成功結果に必要な代数条件をすべて検査するため, この境界は公開 soundness 定理の前提ではない. $S_r = r.\text{state}.\text{prevailing}$ とおくと, 機械化した公開 soundness は次の, より強い形である.

$$\frac{\text{infer}(\widehat{\Sigma}, \Gamma, e) = \text{some } r}{\widehat{\Sigma}; S_r \Gamma \vdash e : r.\text{resolvedTarget}}$$

ここで $r.\text{resolvedTarget}$ は S_r を raw result target に適用した型である. raw traversal と validator はともに停止する executable function であり, 定理の premise には caller-supplied bridge, 型付け oracle, completeness, full principal-type theorem を含めない.

式 checking と application の引数 checking は同じ期待型整合 helper を使う. raw synthesized type が matcher の product で期待型が matcher または matcher slot なら COERCE-PRODUCT-MATCHER を, raw type が slot の product で期待型が matcher slot なら COERCE-SLOT-TUPLE を決定的な branch として選び, その後に通常の type equality または producer-stable な slot alignment を行う. 空 product は両 recognizer に一致するため, product-matcher branch を優先する. これは executable selector の決定性であり, surface coercion 全体の一意性ではない. terminal reconstruction は同じ raw type と期待型から unary coercion node を再構成するため, trace に型

付け oracle を追加しない. raw metavariable が prevailing substitution 後に初めて product head または matcher / slot component を持つ場合の completeness は, matcher product と slot product の双方について現段階では主張しない.

この公開境界には positive / negative control twin を置く. 同じ structured consumer に対して polymorphic value producer を用いる項は infer が拒否し, あらかじめ concrete capability を持つ producer へ置き換えた項は成功して結果型を Int に固定し, 上の定理から HasTy を再構成する. また再帰的な list matcher を実際の fix で構成して slot に適用し, cons \$x \$rest の両 binding を body で使う matchAll が公開 infer を通ること, 結果型の固定, HasTy の再構成を検査する. 後者は静的 inference 回帰であり, この再帰 matcher の動的実行を主張しない.

singleton direct-self の fix では, 再帰 binder は RHS 内で一つの monotype placeholder を共有し, 公開 W も T-FIX と同じ DirectSelf 境界を検査する. actual clause evidence 以外の self-reference や demand は ShapeCap の seed にしない.

7.3 Concrete runtime safety

runtime matcher value は source の全 clause 列と current clause/arm cursor を別々に保持する. 公開 matching-state 境界では, 値の内部, closure と matcher の captured environment, 内部 node, stack の全 cursor が original clause 列にあることを Pristine とする. suffix cursor は一つの MAtom 導出内だけで使い, successor state へ流出させない. 整型 matcher value からは元の actual clause typing, evidence, CoverageOK を復元できる. matching state typing は atom ごとの prevailing substitution, pattern-function node の固定 parameter context, 残余 actual argument, 内部 stack, terminal substitution の binding 順序を追跡する.

runtime の CapabilityDemand は, raw DemandMatches または検査済み matcher-to-slot certificate から得られる, 正規化済み producer / consumer endpoint compatibility の sound な忘却である. raw consumer 構文と一つの substitution が表す共有相関そのものは index に持たない. 同じ raw consumer variable の二度目を wildcard として扱わない strict check は忘却前の DemandMatches / raw certificate が行い, CapabilityDemand から exact raw origin を復元する converse は主張しない.

Ordered clause dispatch では, original clause 列から現在位置までの失敗 prefix を DispatchTrace が保持する. freeze 済み signature は compatible な child capability 列の一意性と, 各 compatible pattern-constructor instance の canonical former / arity 登録を要求する. 従って最後の bare-hole catch-all まで到達した structured pattern は, 先行する対応 constructor clause の失敗事実と矛盾し, stuck にならない.

値パターン式 #e が原子へ入る時点の context で型付くことを CaptureAdm とする. これは caller-supplied premise ではなく, clause typing の PPatCoreOrder, PP typing, user-pattern typing, 成功した concrete PPM 導出から機械的に導く. 順序条件は, 同じ PP で左側の hole が導入した binding を #e が参照する場合を排除する. StepReady は一段の clause dispatch に必要な body / next-matcher の局所評価結果だけを持ち, 一般の program termination や既成の Step 導出を仮定しない.

freeze 済み signature の整形式性と source/runtime pattern-function signature の一致の下で, Lean の concrete judgments 上に次を証明する.

1. pristine な typed environment ρ での $\text{Eval } \rho \ e \ v$ は HasTy を ValueTy へ保存する.
2. 整型 matching state の一段 Step の全 successor は整型である.
3. 非終端の整型 state は, 局所証拠 StepReady の下で一段進む. 空 successor 列は正当な match failure であり stuck ではない. これは型付けだけから一段を発見する古典的 progress ではなく, 規則ごとの局所実行証拠から concrete step を組み立てる定理である.
4. 一段保存の反復により, 到達可能な全 state は整型である.
5. 空 stack へ到達した成功 branch の substitution は, source pattern が生成した binding context で型付く.
6. matcher consistency は前項を初期 atom からの到達へ適用した系として得る.

最後に W の再構成定理をこの declarative safety package へ接続する. caller-supplied runtime agreement は derivation-local mirror を別の oracle として与える必要はない. 全 source context Γ に対する $\text{RuntimeSigAgrees}(\hat{\Sigma}, \Gamma, SF)$ から, 任意の concrete Eval / PPM / MAtom / Step / Search / Reaches 導出に対応する mirror を, その導出の構造再帰で構成する bridge を機械化する. この bridge は評価結果, successor, 停止性, 型付け導出を追加前提としない.

また $SF = []$ の concrete frozen signature と一つの実際の `matchAll` について, `FrozenSigWF`, global runtime agreement, 評価, primitive-pattern matching, atom reduction, 二段の state step, search, reachability と全 mirror を同時に構成する. この最初の end-to-end regression が具体的に消費する `CoreSafety` field は evaluation preservation であり, 公開 W の結果と保存後の runtime value をともに `List Int` へ固定する.

別の capture regression は, 合法的な value-pattern-pattern $\#x$ に対して `PPM.pval` から `MAtom.matcher`, state step, search, reachability を具体的に構成する. `AtomTy.mk` から得た初期 state へ step preservation を適用するため, `captureAdm_of_coreOrder` の value-pattern 分岐が実際の reduction で消費される.

ordered-dispatch regression は具体的な `cons $x $rest` を, 先行する `nil` clause の primitive-pattern failure, `cons` clause の先頭 data-pattern arm failure, 次 arm の成功へ順に通す. 一つの kernel 導出内で `MAtom.matcherPPFail`, `MAtom.matcherDPFail`, 二 child の `PPM.ctor`, `MAtom.matcher` を発火させ, 同じ証拠から `DispatchTrace.nextClause` と `DispatchTrace.nextArm` を構成する. 生成された二 atom は x と $rest$ を束縛し, state step, search, reachability, `matchAll` 評価を terminal まで閉じる. さらに同じ private cursor の三段遷移を, 非再帰 Pair α family の型付き fixture でも発火させる. 後者は `frozenSigWFCheck` と公開 W を通り, 非空な $[(rest, 2), (x, 1)]$ に到達する同一実行へ evaluation / step preservation, reachability, search substitution typing, matcher consistency を適用する.

さらに source/runtime signature が closed nullary pattern function `unit := ptuple []` を一つ持つ非空の例を置く. 全 context に対する global agreement から必要な各 mirror を構成し, 非空な `RuntimeSigAgrees.sourceLookup` も step preservation と local progress で実際に消費する. `papp` による node 進入, inner tuple reduction, completed-node removal, terminal search までを実行する. この同じ fixture は `CoreSafety` の六 field をすべて具体的に適用し, search substitution と matcher consistency には実際の $[] \in [[]]$ の member を渡す. 公開 W の結果は `List Int` に固定する. 従って六つの conclusion は個別に存在するだけでなく, 一つの非空 runtime-signature program 上で同時に発火する. 加えて非 nullary の `pass(parameter : Unit) := embed parameter` について, context ごとに fresh な binder を選ぶ局所 scheme と固定 lookup scheme の $\equiv_{\text{vf}}$ を構成する. これにより全 context に対する runtime agreement と, 実際の `patfunEnter` step に対する global bridge の mirror が引数付き定義でも成立する.

公開 composition の境界には `FrozenSigWF` と適切な runtime agreement が残るが, 局所的な実行証拠として caller が与えるのは `StepReady` だけである. さらに `FrozenSigWF` は裸の仮定ではなく, 有限の実行可能検査 `frozenSigWFCheck` の成功から定理 `frozenSigWFCheck_sound` が構成する. 検査は fail closed であり, canonical な `nil` / `cons` 宣言, data constructor の構文的 data root と binder 被覆 (instantiation の一意性は substitution-agreement の逆補題から従う), 単一パラメータ collection family に限定した pattern constructor (capability projection の明示的 inversion が uniqueness と coverage-index coherence を与える) を検査する. 各 pattern constructor には `constructorsByFormer` の対応 row と $(name, arity)$ membership を必須とし, row 欠落は fail closed で拒否する. さらに canonical scheme に一致する primitive (delta preservation は operation ごとに一度証明する), 非重複な pattern-function 名前空間を検査する. 関数値 field `armExhaustive = basicArmExhaustive` だけは signature 構成時の定義等式として受け取る. 空でない pattern-constructor 表を持つ list / multiset signature の `FrozenSigWF` もこの checker の一回の実行で立つ. 対応 row を削除した List signature は checker が拒否し, join 一般節を欠く multiset matcher は `coverageCheck = false`, $\neg \text{CoverageOK}$, raw / 公開 W のすべてで拒否される.

pattern を含まない λ / `let` / direct-self `fix` 断片については, 独立に定義した一 sort の Damas-Milner 体系からの埋め込み定理を機械化する. その再帰規則も $f \neq x$ と `DirectSelf(f, e)` を要求し, この制限体系のすべての導出は閉じた frozen signature ($\text{ftv} = []$) 上の二 sort 宣言体系の導出へ写る. 埋め込みの下で capability binder は空であり, capability substitution は埋め込まれた型に自明に作用し, `let` の一般化は二 sort generalizer と可換である. 逆方向 (conservativity) と algorithmic acceptance は主張しない.

principality はそのままの形では成り立たない。閉じた式 (something, something) は T-TUPLE により matcher の product 型へ、COERCE-PRODUCT-MATCHER により product matcher 型へ型付くが、tuple 式の導出可能型の頭構成子は prod / matcher / slot に限られ、paired substitution はこれらの頭を保存する。従って両方の型付けを instance に持つ導出可能型は存在しない。この反例は機械化済みであり、原因は coercion の重なりそのものであって capability evidence とは独立である。一方、surface typing を非 coercion root と明示的な外側 coercion spine へ分解する定理、および公開 inference の成功から derivation-structured な Prop 証明書として core typing を構成し、それを erase して surface typing を得る定理を機械化している。proof irrelevance のため、この証明書自体は観測可能な elaboration data ではない。現段階の surface 分解は再帰 premise に surface typing を残す root factorization である。一方、inference が構成する core typing 証明書は pattern の raw expression / parameter context を全 child で共有し、raw binding context を左から右へ thread する。value-pattern, arm, clause の premise も再帰的な core evidence であり、surface typing oracle へ戻らない。この証明書はさらに、再帰 premise を core evidence に保つ non-coercion head と明示的な外側 coercion plan に分解でき、逆向きの再構成も可能である。ただし core typing の一意性と surface typing に対する再帰的 completeness はまだ主張しない (unifier の置換普遍性は機械化済みである)。一方、命題値の coercion reachability とは別に、型の頭を変える 3 つの observable primitive step, 非空 spine, identity を持つが一般の trans を持たない candidate normal-plan syntax を観測可能な型として定義する。product of matchers から slot への指定二段経路は COERCE-PRODUCT-MATCHER の後に COERCE-MATCHER-TO-SLOT, product of slots から aggregate slot への経路は非空 product に対する COERCE-SLOT-TUPLE 一段とする。surface の slot-to-slot check は capability / target の決定的 unifier と後置換の後で両端が等しいことを証明し、identity へ吸収する。全 observable step / spine が端点を変えること、同じ端点の plan が identity だけであること、既存 surface coercion および typing への sound replay を証明する。identity 以外で合成可能な唯一の二段経路を whole-product-first spine へ畳み、空 product の slot-tuple rule も matcher-first の二段経路へ送ることで、任意の surface plan が candidate normal plan を持つことを証明する。ただし surface plan は命題値なので、結果は観測可能な値ではなく Nonempty に包まれた論理的存在である。同じ端点に対する observable rule 列の一意性も証明する。raw certificate を含む plan inhabitant 自体の一意性と、公開推論結果への plan data の格納はまだ主張しない。

capability 後置換には rigid / rename-only / structural-flexible を区別する有限 ledger を用いる。admissible post は逐次合成で閉じ、freeze 後に既存の variable-only 境界へ接続できる。phased post は局所 structural phase と frozen residual phase を分離する。制約対象は capability component だけで、target component は無制約である。W state の shadow ledger は一般 fresh capability と constructor / primitive image を structural-flexible, scheme image と finalized producer を rename-only と記録するが、solver と terminal audit は従来の protected-producer 検査を維持する。origin-aware solver の kernel slice は機械化済みである：paired solver は型構造を再帰しながら capability / target の二 sort を同時に解き、matcher / slot 注釈は ledger で方向付けた capability solver (rename-only は構造化禁止・rename 像は非 flexible 限定・structural-flexible は構造化許可・未登録は rigid) へ送る。全成功は soundness と admissible-post 準拠を運ぶ proof-carrying certificate であり、二 sort 合成は逐次合成の閉性による。対称 solver が rigid 注釈比較で拒否する制約を flexible ledger の下で解く対照回帰も固定した。W への切替には cut ごとの snapshot, および raw binder でなく局所 solve 後に外へ残る prevailing-image leaf を freeze する export event が残る。

terminal pattern leaf の任意の actual context をまず actualContext = S rawContext に限定し、leaf-local な coherent judgment, surface への忘却, reconstruction bridge を機械化する。さらに一つの raw expression / parameter context を全 child で共有し、raw binding context だけを左から右へ thread する強い judgment と忘却写像を与える。reconstruction judgment 自体も同じ raw indices を保持する形へ強化し、W の reconstruction motive がこの evidence を直接生成する。value-pattern 内の式 typing は reconstruction 側では再帰的 core evidence であり、surface judgment への bridge でのみ通常の typing へ忘却する。この上に、expression, arm, clause まで含む coherent surface typing を、reconstruction certificate そのものとの定義的同一視として公開する (鏡写しの独立 domain は維持しない)。pattern 層は threaded 形、value-pattern / arm body / clause next-matcher の式 premise は certificate 自身へ再帰し、surface typing への忘却と、公開 inference の成功から coherent typing を得る系を持つ。product lift の構成子は将来の raw-head 可視 fragment を言明するための raw-source provenance 添字を certificate 本体に保持し、W の再構成は selector が実際に頭を検査した raw type を faithful に注入する (恒等 witness も常に取れるため判断

を制限しない). coherence が制限するのは pattern の provenance だけなので, `matchAll` と `matcher literal` を含まない match-free 断片では任意の surface typing が coherent であり, Damas – Milner 断片の宣言的型付けはすべて埋め込みを経て coherent judgment に入る (DM 受理目標の宣言側). 最上位目標は閉じたプログラムに対する受理完全性 (annotation-freeness) $\forall e \tau, \emptyset \vdash e : \tau \Rightarrow \text{infer}(e) \neq \text{none}$ である. 言明を固定した上で, 推論器側の修正を段階的に積んで成立させる到達目標として扱う (or pattern の binder 整合は解消済み・受理側 regression で固定). なお or の certificate (deriv / threaded の or 規則) は左右の raw 結果を別々に持ち prevailing 像の等価 premise で結ぶ形へ緩和したが, 宣言的 terminal or 規則は不変である. COERCE-PRODUCT-MATCHER は tuple literal 専用ではなく, product-of-matchers 型を持つ任意の式への unary lift であるため, `let` 後の変数利用位置にも明示的 coercion を挿入できる. 期待型整合 helper は raw product-of-matchers に加えて raw product-of-slots も認識し, 後者を slot として使う場合には COERCE-SLOT-TUPLE を選ぶ. 通常の application も function を fresh domain / codomain へ先に整合してから argument をその domain に対して check するため, 関数引数位置で product-matcher lift, matcher-to-slot, slot-tuple を挿入できる. ただし raw metavariable が prevailing substitution 後に初めて product head または matcher / slot component を持つ場合には, matcher product / slot product の双方で, 判明した solve cut と後続 suffix を記録する cut-indexed event がなお必要である. 空 product は両 recognizer に一致するため product-matcher branch を優先する. 外側 plan から candidate syntax への論理的 normalization completeness は得られた. この先は, component-first 経路を含む critical pair の解消, full plan uniqueness (または適切な quotient), coercion の意味的同値, 置換に対する naturality を確定してから, core typing の一意性と coherent surface completeness へ進む必要がある. 二 sort unifier の最汎性 (返された substitution を任意の unifier が factor すること) は proof-carrying kernel の certificate として, fuel 単調性 (成功が任意のより大きい fuel で同じ substitution のまま保存されること) は fuel の直接帰納として, $\exists \text{fuel}$ solvability completeness (可解な制約はある fuel で成功すること) は「残り変数 budget と構造 weight」の辞書式 well-founded 帰納と, 真に不等な head の解が budget 変数一つを消去する variable-elimination certificate として機械化済みであり, 残るのは固定の構造 fuel bound を使う公開 wrapper の十分性である. capture admissibility, 任意の capability 輸送, 抽象 runtime oracle, 型付け済み外部 list / multiset matcher は不要である. Lean の主定理と一般補題は `sorry`, `admit`, `project-defined axiom` を用いず, kernel が検査する. `#guard` / kernel reduction で評価する小さな回帰も同様である. 一部の大きな executable regression で用いる `native_decide` だけは, その計算結果について Lean の native compiler を追加で信頼する.